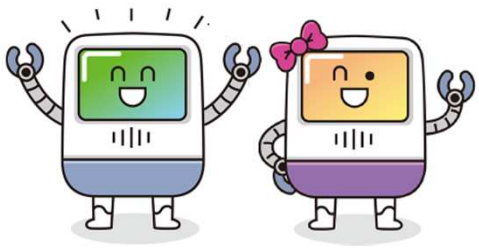
딥러닝

3장 개발환경 구축과 넘파이



03

개발환경 구축과 넘파이

학습목차

01 파이썬과 구글 코랩
02 구글 코랩의 여러 가지 기능들
03 강력한 기능을 가진 넘파이를 알아보자
04 넘파이의 고급 기능을 알아보자
05 딥러닝을 위한 행렬

학습목표

- 파이썬 언어와 딥러닝 라이브러리인 텐서플로를 코랩 환경에서 사용하는 방법을 익힌다.
- 넘파이의 특징과 브로드캐스팅, 벡터화 연산의 장점을 이해한다.
- 넘파이의 다양한 기능을 알아본다.
- 딥러닝을 위한 행렬의 핵심 기능을 알아본다.
- 인공신경망을 구현하기 위하여 행렬을 사용하는 이유를 알아본다.



- 파이썬과 구글 코랩
  - 파이썬 프로그래밍 언어
  - 코랩 환경에서 코딩하기
- 구글 코랩의 여러 가지 기능들
  - 코랩과 가상환경
  - 딥러닝을 위한 라이브러리 살펴보기
- 강력한 기능을 가진 넘파이를 알아보자
  - 다차원 배열의 속성을 알아보자
  - 파이썬의 강력한 자료구조인 리스트와 넘파이의 다차원 배열
  - 다차원 배열의 연산
  - 편리하고 강력한 브로드캐스팅과 벡터화 연산
  - 다차원 배열을 쉽게 생성해 보자
  - 다차원 배열을 합하는 `append()` 함수
- 넘파이의 고급 기능을 알아보자
  - 인덱싱과 슬라이싱
  - 난수를 생성해 보자
  - 여러 개의 배열값을 얻는 방법
  - 배열의 형태를 바꾸는 `reshape()` 메소드와 `flatten()` 메소드
- 딥러닝을 위한 행렬
  - 딥러닝을 위한 행렬의 핵심 개념
  - 행렬 곱 연산
  - 인공신경망과 행렬
  - 단위행렬과의 곱

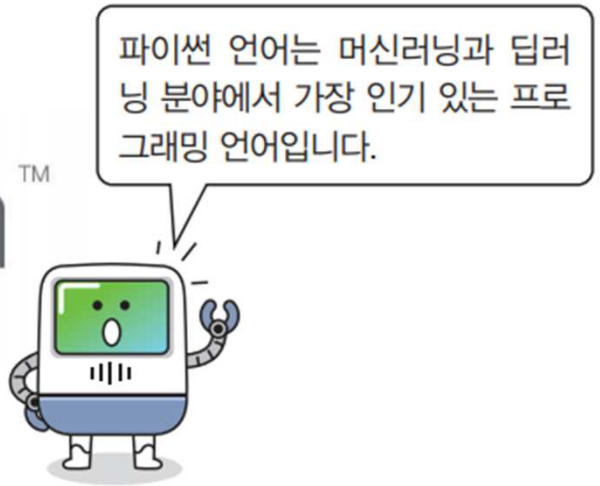
- 파이썬 프로그래밍 언어
  - 이 책에서는 인공지능 기술 중에서도 딥러닝과 관련된 라이브러리인 **텐서플로** tensorflow 를 사용할 것이며, 프로그래밍 언어는 **파이썬** python 을 사용할 것이다
  - 파이썬은 귀도 반 로섬이 만든 프로그래밍 언어이다



파이썬 언어의 창시자
귀도 반 로섬

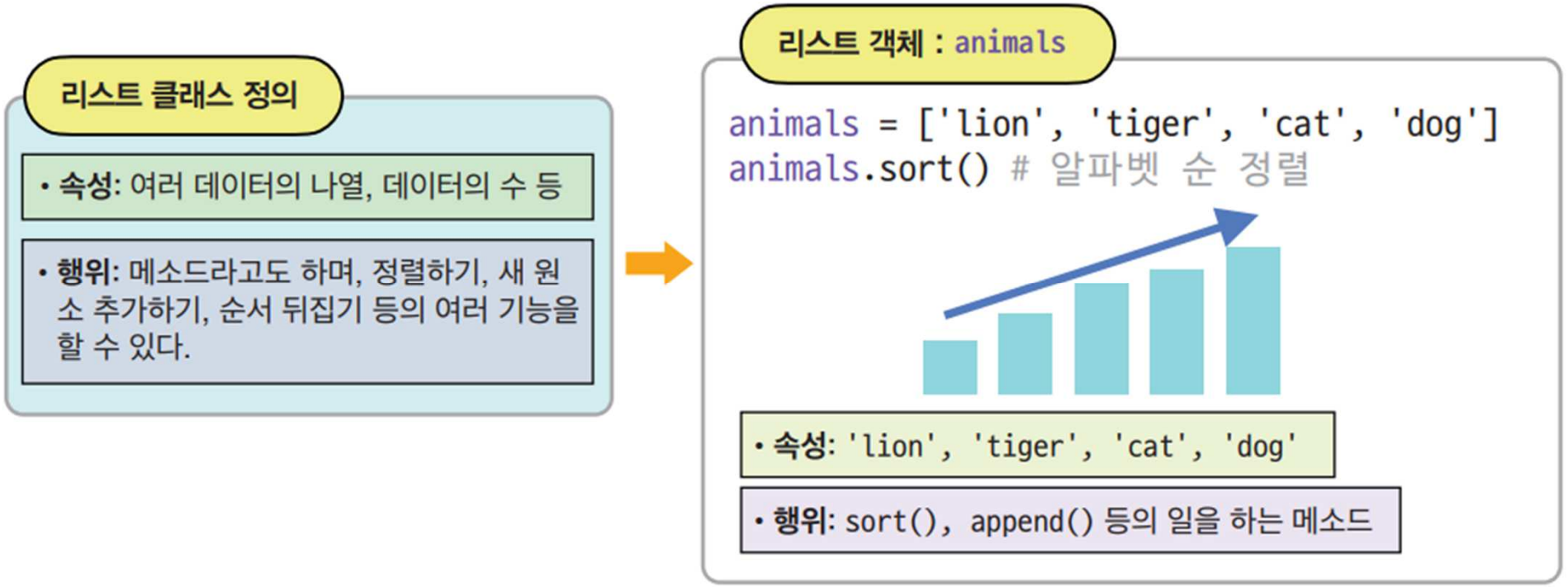
python™

파이썬 언어의 로고



- 파이썬 프로그래밍 언어
  - 정수, 부동소수점,문자와 같은 전통적인 **자료형** data type 뿐만 아니라, 데이터 묶음을 처리하기에 편리한 리스트, 튜플,딕셔너리, 집합과 같은 자료형을 기본으로 제공한다는 점이다.
  - 리스트와 튜플 자료형은 데이터에 접근하기 위해 전통적으로 배열에서 사용하는 **인덱싱** indexing 방식을 사용할 수 있으며, 전체 데이터의 일부분을 편리하게 잘라낼 수 있는 **슬라이싱** slicing 기능은 파이썬을 이용한 데이터 처리를 더욱 효율적으로 만들어 준다
  - 프로그램의 복잡도가 높아지면 코드의 관리가 매우 중요한데, 가장 뛰어난 해결책으로 인정받고 있는 방법이 바로 **객체 지향 프로그래밍** object oriented programming 방식이다
  - 파이썬은 '객체 지향 프로그래밍 언어'이며, 파이썬이 다루는 모든 자료형, 함수, 모듈은 객체이다.

- 리스트 클래스의 정의와 리스트 객체의 관계
  - 클래스는 객체를 만들기 위한 틀이다.
  - 클래스는 속성과 행위(메소드)를 정의하고 있다.



- 파이썬 프로그래밍의 예시
  - animals 객체를 생성하고 animals.sort() 라는 메소드를 호출하여 알파벳 순으로 정렬이 가능함.

```
>>> animals = ['lion', 'tiger', 'cat', 'dog']
>>> animals.sort()              # animals 리스트 내부 문자열을 알파벳 순으로 정렬
>>> animals
    ['cat', 'dog', 'lion', 'tiger']
>>> animals.append('rabbit')  # animals 리스트에 새 원소를 추가
>>> animals
    ['cat', 'dog', 'lion', 'tiger', 'rabbit']
>>> animals.reverse()           # animals 리스트를 원래 원소의 역순으로 재배열
>>> animals
    ['rabbit', 'tiger', 'lion', 'dog', 'cat']
```

코랩 환경에서 코딩하기

- 이 책의 실습은 **코랩** Colab 환경에서 파이썬으로 이루어지며, **넘파이** NumPy, **판다스** pandas 와 같은 데이터 처리 패키지와 **사이킷런** scikit-learn, **텐서플로** TensorFlow 와 같은 머신러닝 도구를 사용할 것이다.
- 코랩 환경에 접속하는 주소(반드시 구글 계정이 있어야 함)

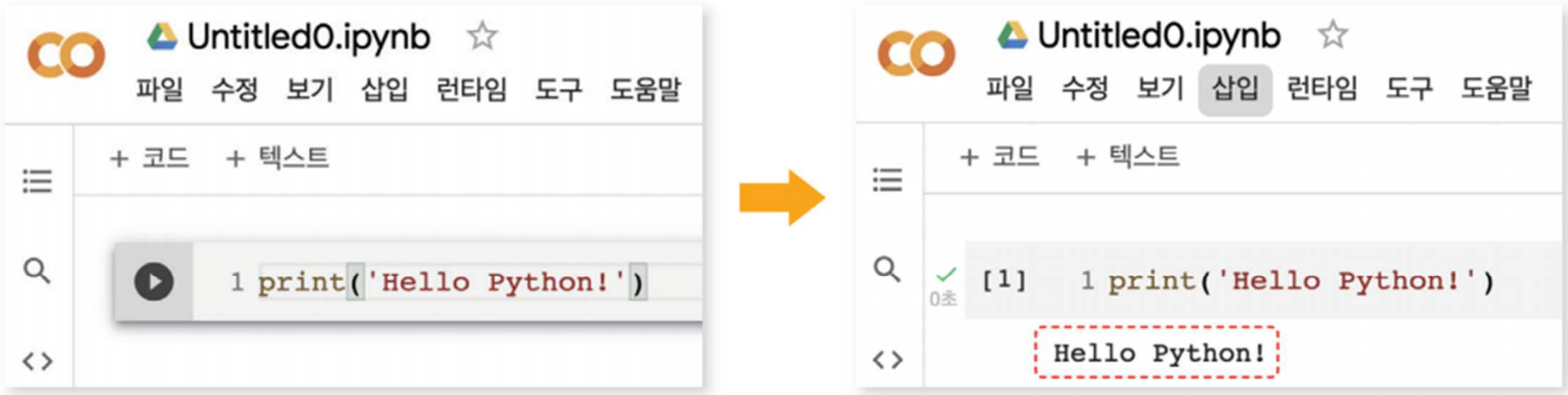
```
https://colab.research.google.com/
```

- 다음과 같은 장점이 있다.

- 유료로 이용할 수 있는 고급 서비스가 있으나 대부분의 기능을 무료로 이용할 수 있다.
- 클라우드 기반의 서비스이므로 파이썬을 따로 설치하지 않고도 웹 환경에서 파이썬 사용이 가능하다.
- 다른 개발자들과의 파일 공유가 가능하며 협업을 통한 개발도 손쉽게 할 수 있다.
- 넘파이, 판다스, 사이킷런, 텐서플로 등의 패키지가 미리 설치되어 있다.
- 클라우드에서 환경에서 제공하는 GPU[3]와 TPU[4]를 사용하여 고성능 계산을 빠르게 할 수 있다.

코랩 환경에서 코딩하기

- 구글 계정으로 로그인하고 새로운 파이썬 파일인 파이썬 노트북을 생성해 보도록 하자.
  - 이를 위하여 파일 메뉴의 새 노트를 선택하고 다음과 같은 코딩을 하자.



코드 셀에 파이썬 명령을 입력하고 Shift + Enter 키로 실행시키면 아래에 실행 결과가 나타난다.

코랩 환경에서 코딩하기

- 작성 파일의 이름은 Untitled0.ipynb 파일과 같으며 수정할 수 있다.
- 코드의 실행 순서도 [1], [2]와 같이 볼 수 있다.

```
print('Welcome to Deep Learning!')
```

```
Welcome to Deep Learning!
```



코랩 환경에서 코딩하기

도전 문제 3.1: 코랩에서 파이썬 코딩하기

상 중 하



1. 다음과 같은 출력을 내보내는 파이썬 코드를 작성하여라.

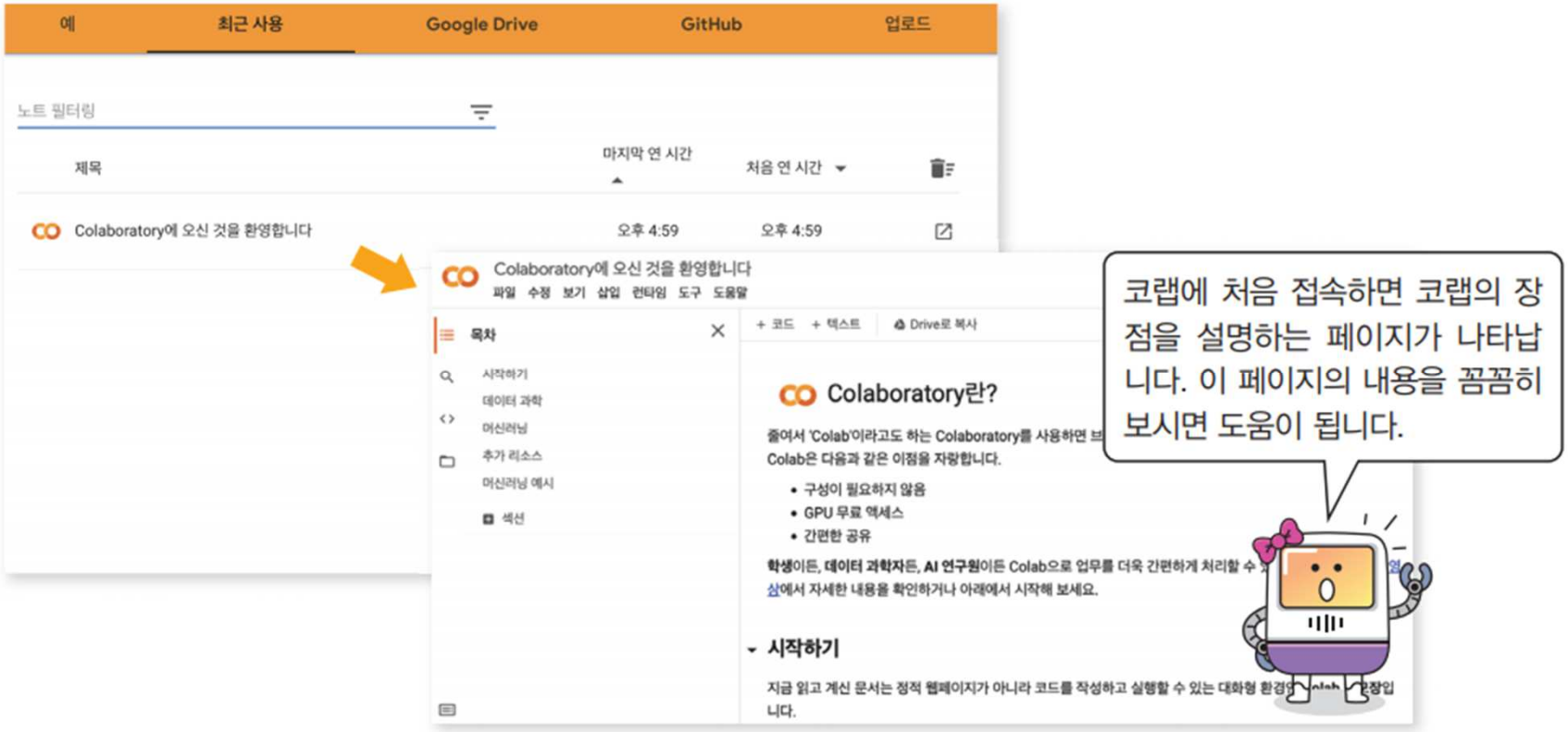
```
안녕하세요 딥러닝의 세계로 들어온 여러분을 환영합니다!
```

2. 다음과 같이 여러분의 이름, 학년, 소속, 취미를 출력하는 파이썬 코드를 작성하여라.

```
이름 : 홍길동
학년 : 2학년
소속 : 대한대학교 정보통신공학과
취미 : 서핑, 수영
```

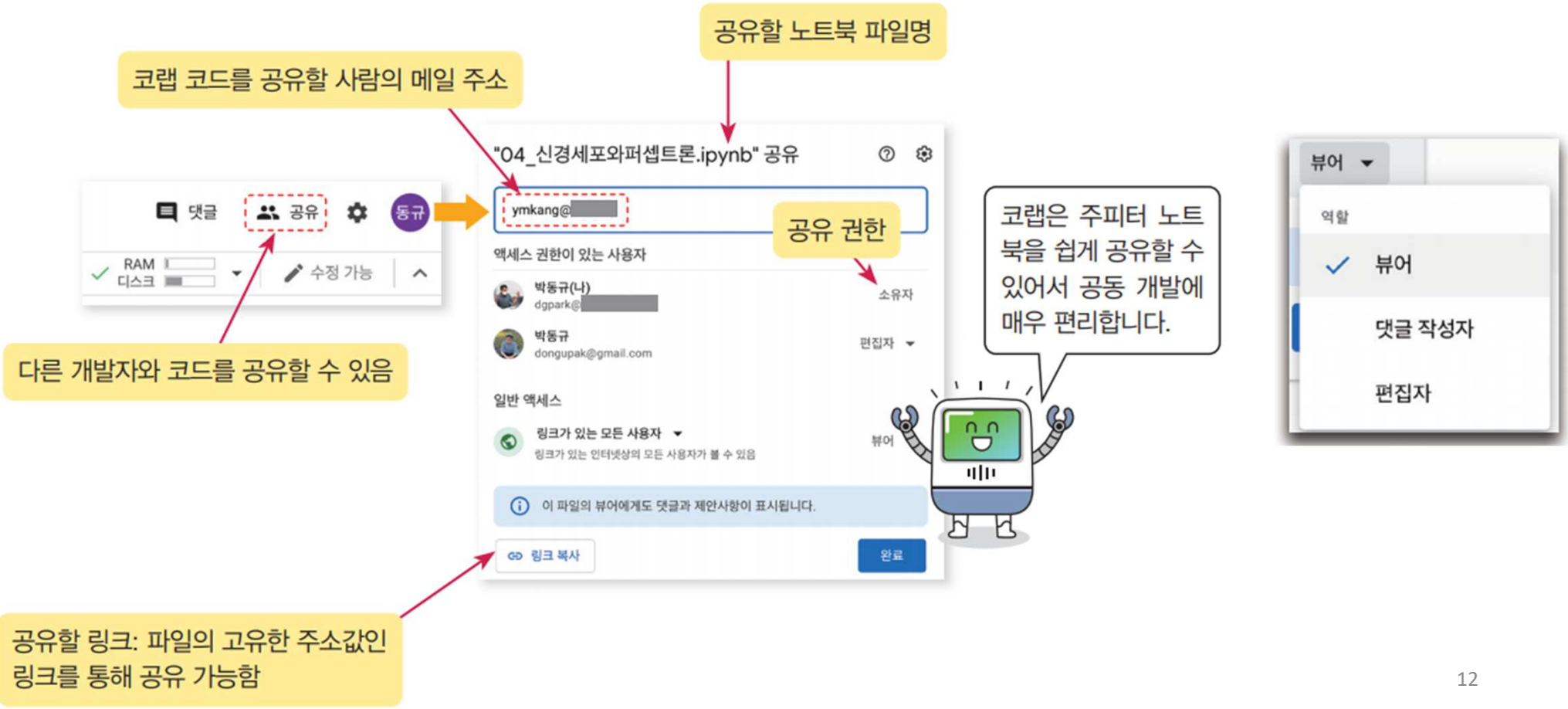
02 구글 코랩의 여러 가지 기능들

- 코랩과 가상환경
  - 구글 드라이브와의 연동 기능, 편리한 노트북 파일의 공유 기능 등이 제공됨



코랩과 가상환경

- 손쉬운 공유와 공동 개발, 작업이 가능하다.
  - 다음 작업을 해보자.



코랩과 가상환경

도전 문제 3.2: 다음 작업을 코랩 환경에서 수행하자

상 중 하



1. `Hello World!`를 출력하는 파이썬 프로그램을 코랩 환경에서 작성하자.
2. 이 파일의 이름을 `hello.ipynb`로 변경하자.
3. `hello.ipynb`를 동료 작업자와 공유하도록 하자. 공유 권한은 편집자로 지정하자.
4. 파일을 공유받은 동료 작업자는 이 코드 다음에 `Hello Deep Learning!`을 출력하는 코드를 작성하자.

딥러닝을 위한 라이브러리 살펴보기

- **텐서플로** Tensorflow 는 머신러닝과 딥러닝을 위한 오픈 소스 플랫폼으로 구글의 인공지능 개발부서에서 개발하여 내부적으로 사용하다가 2015년에 오픈 소스로 공개되어 현재 텐서플로 2 버전으로 발전하였다.

  - 구글사에서 공식적으로 지원하고 배포하는 딥러닝 라이브러리이다.
  - 강력한 다중 그래픽 프로세서 유닛(GPU)을 지원한다. 딥러닝을 위해서는 많은 연산이 필요하다. 만일 강력한 그래픽 프로세서 유닛이 있을 경우, 이러한 딥러닝 연산을 빠르게 수행할 수 있다. 텐서플로에서는 코드를 수정하지 않고도 CPU나 GPU에서 모델을 실행시키는 것이 가능하다.
  - 텐서플로는 텐서보드라는 기능으로 그래프 시각화를 지원한다. 이 기능을 통해서 학습이 이루어지는 과정을 쉽게 살펴볼 수 있다.
  - 텐서플로는 많은 사용자를 확보하고 있어서 다양한 문서와 활발한 인터넷 커뮤니티가 존재한다. 이 때문에 필요한 정보를 빠르게 찾을 수 있다.

딥러닝을 위한 라이브러리 살펴보기

▪ 파이토치 알아보기

- 파이토치는 토치라는 딥러닝 라이브러리에 기반하고 있는데, 이 기능을 파이썬으로 이용할 수 있도록 만든 것이 PyTorch이다.
- 페이스북의 인공지능 연구 그룹에서 주도적으로 개발하는 오픈 소스 라이브러리이다.
- 다중 GPU의 지원을 받는 병렬 프로그램을 쉽게 구현할 수 있다.
- 딥러닝 모델을 만들기 위한 유연하면서도 직관적인 인터페이스를 제공하고 있기 때문에 사용하기 쉽다.
- 외부 라이브러리를 사용해서 데이터 교환을 쉽게 할 수 있는 기능을 제공한다.

딥러닝을 위한 라이브러리 살펴보기

- 다양한 머신러닝, 딥러닝 플랫폼이 있다.



딥러닝을 위한 라이브러리 살펴보기

- 이 책은 텐서플로 2 버전의 기능을 중심으로 살펴볼 예정이다. 텐서플로 모듈의 이름은 tensorflow이며 tf라는 별명을 주로 사용한다.
  - 텐서플로의 버전은 다음과 같은 __version__ 속성을 통해 확인할 수 있다

```
import tensorflow as tf          # tensorflow의 별명은 tf로 한다
print(tf.__version__)            # version 앞뒤로 두 개의 밑줄을 사용한다
```

```
2.13.0
```

도전 문제 3.3: 머신러닝과 딥러닝 라이브러리

상 중 하



1. 테아노, mxnet, 사이킷런 라이브러리의 중요한 특징과 기능에 대해서 정리해 보자.
2. 위에서 언급한 내용 이외의 텐서플로 특징을 조사하고 정리해 보자.

03 강력한 기능을 가진 넘파이를 알아보자

- 넘파이는 파이썬에서 수치 데이터를 다루는 가장 기본적이고 강력한 패키지이다.
  - 데이터 분석이나 머신러닝 프로젝트를 수행한다면 넘파이에 대한 확실한 이해가 필수적

1. **강력한 다차원 배열 처리:** 넘파이는 다차원 배열 객체인 ndarray를 제공한다. 이를 통해 효과적으로 다차원 데이터를 처리할 수 있으며, 이는 과학 계산 작업이나 머신러닝 알고리즘에서 필수적이다.
2. **빠른 연산 속도:** 넘파이는 내부적으로 C언어로 구현되어 있어, 파이썬 내장 리스트에 비해 빠른 연산 속도를 보여준다. 이 기능으로 인해 대량의 데이터를 처리할 때 효율성이 크게 향상된다.
3. **다양한 수학 함수를 제공:** 넘파이는 간단한 사칙연산부터 난수 생성 등과 같은 복잡한 수학 함수까지 다양한 함수를 제공한다. 또한, 이러한 함수들은 배열의 각 요소에 자동으로 적용되는 벡터화 연산을 지원한다.
4. **편리한 브로드캐스팅 기능:** 넘파이의 브로드캐스팅 기능은 서로 다른 크기의 배열 간에도 산술 연산을 가능하게 한다. 이는 코드를 매우 직관적이고도 간결하게 만들어주는 효과가 있다.
5. **다른 라이브러리와의 호환성:** 넘파이는 파이썬 데이터 과학 생태계의 핵심으로, 판다스, 맷플롯립, 사이킷런, 텐서플로 등 다른 많은 라이브러리들이 넘파이 배열을 기반으로 동작한다.

03 강력한 기능을 가진 넘파이를 알아보자

- 코랩 환경에서는 넘파이 패키지가 미리 설치되어 있다.
  - 하지만 일반적인 경우 윈도우나 리눅스, 맥 운영체제의 명령 프롬프트상에서 pip를 이용하여 설치하는 과정이 필요하다.
  - pip는 파이썬의 패키지 관리 소프트웨어로, 파이썬 표준 라이브러리에 포함되지 않은 외부 라이브러리를 설치하도록 도와주는 도구이다.
  - 다음과 같은 운영체제의 명령행 명령으로 넘파이를 설치할 수 있다.

```
pip install numpy
```

  - 넘파이 설치 후 간단한 다차원 배열 객체를 생성하는 명령

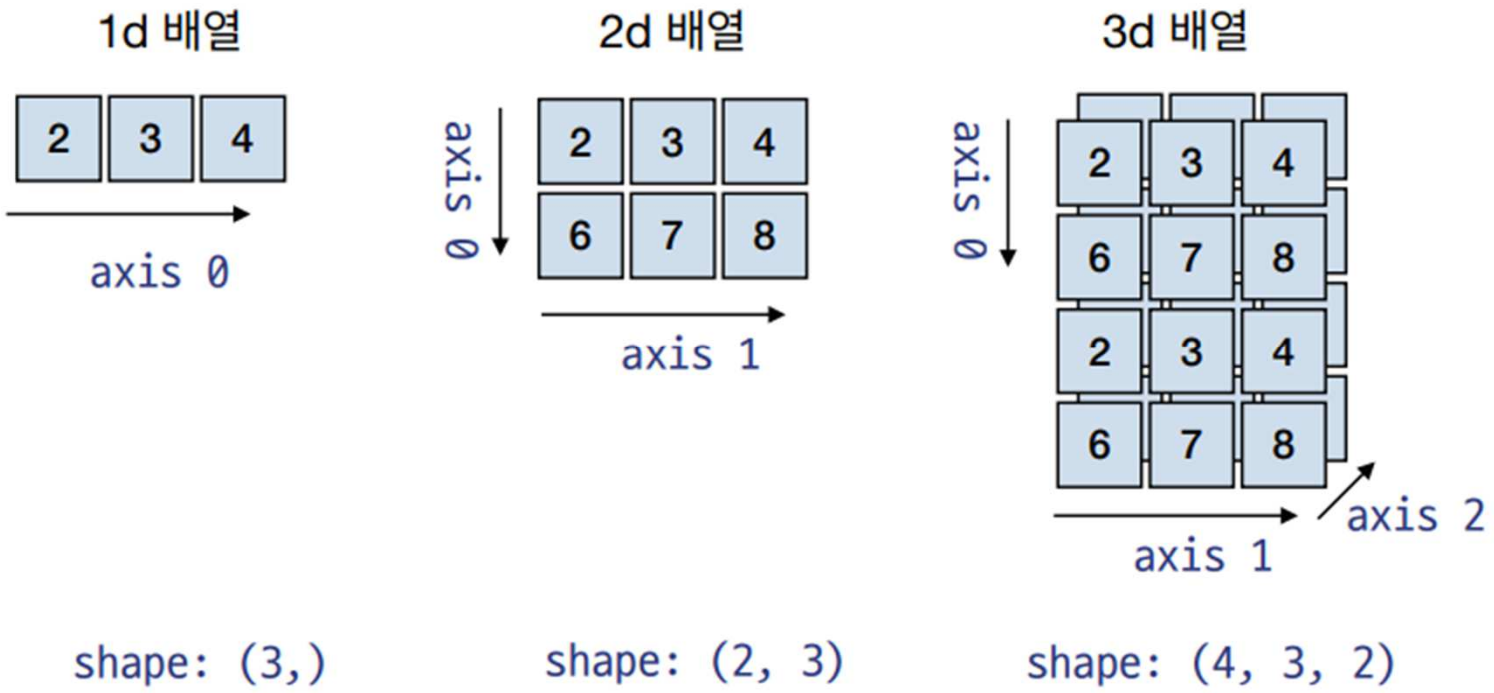


```
import numpy as np          # numpy의 별칭으로 np를 지정함
a = np.array([2, 3, 4])     # 다차원 배열 객체
```

03 강력한 기능을 가진 넘파이를 알아보자

- 배열은 2, 3, 4의 정수 값으로 구성되어 있는데 이 개별적인 값을 항목 item 또는 **원소** element 라고 한다.
  - 이 연속적인 원소들을 가지는 객체가 바로 넘파이의 핵심이 되는 다차원 배열 객체이다.

배열 내부의 각 요소는 **인덱스** index 라고 불리는 정수들로 참조된다. 또한 넘파이에서 차원은 **축** axis 이라고도 한다.
배열의 모습을 **형상**shape이라 한다.



- 넘파이와 ndarray

NOTE: 넘파이의 사용과 ndarray의 특징



이 책에서 나타나는 **모든 코드에는 기본적으로 다음 문장이 임포트되어 있다고 가정**하도록 한다. 비록 코드가 생략되어 있더라도 이 코드를 여러분이 직접 입력해야만 에러가 없을 것이다.

```
import numpy as np
```

또한, 이 장에서 다룰 넘파이의 핵심 객체인 ndarray는 다음과 같은 주요 특징이 있다.

- ndarray는 속도가 빠른 프로그래밍 언어인 C언어에 기반한 배열 구조로 되어 있어서 **메모리를 적게 차지하고 연산 속도가 매우 빠르다**.
- ndarray를 사용하면 **배열과 배열 간에 수학적인 연산**을 적용할 수 있다.
- ndarray는 **고급 연산자와 풍부한 함수**들을 제공한다.

다차원 배열의 속성을 알아보자

- 넘파이의 핵심이 되는 **다차원 배열** ndarray 은 다음과 같은 속성을 가지고 있다.
  - 아래와 같이 간단한 값을 가지는 다차원 배열 객체 a를 만들고 이 객체의 속성을 출력해 보자

```
a = np.array([2, 3, 4])        # 넘파이 ndarray 객체의 생성
# a 객체의 형상(shape), 차원, 요소의 자료형, 요소의 크기(byte), 요소의 수
a.shape, a.ndim, a.dtype, a.itemsize, a.size
```

```
((3,), 1, dtype('int64'), 8, 3)
```

속 성	설 명
ndim	배열축 혹은 차원의 개수를 나타냄
shape	배열의 형상을 기술하며 (*m*, *n*) 형식의 튜플 형으로 나타냄
size	배열 원소의 개수를 말하며 (*m*, *n*) 형상 배열의 size는 $m \cdot n$ 임
dtype	배열 원소의 자료형을 기술함
itemsize	배열 원소의 크기를 바이트 단위로 기술함 예를 들어 int32 자료형의 크기는 32/8 = 4 바이트가 됨
data	배열의 실제 원소를 포함하고 있는 버퍼임
strides	배열의 차원별로 다음 요소로 점프하는 데에 필요한 거리를 바이트로 표시한 값

다차원 배열의 속성을 알아보자

- 다차원 배열의 행과 열을 비롯한 차원의 형상을 출력하는 shape 속성은 넘파이 이용자들이 가장 자주 사용하는 속성이다.
  - 다음과 같은 명령으로 배열의 shape 속성을 알아보자.

```
b = np.array([[1, 2, 3], [4, 5, 6]])  # 넘파이 ndarray 객체의 b 생성
b.shape
```

```
(2, 3)
```

파이썬의 강력한 자료구조인 리스트와 넘파이의 다차원 배열

- 파이썬 리스트와 넘파이 다차원배열을 비교해 보자.

```
mid_score_lst = [70, 85, 92, 79]
# 파이썬의 리스트를 이용하여 성적을 저장
print('중간시험 점수 :', mid_score_lst)
```

```
중간시험 점수 : [70, 85, 92, 79]
```

```
# 파이썬의 리스트와 연산
student_mid = [70, 85, 92, 79]        # 중간시험 성적
student_fin = [90, 65, 70, 85]        # 기말시험 성적
student_sum = student_mid + student_fin
print('중간시험과 기말시험 성적의 합 :', student_sum)
```

중간시험과 기말시험의 합이 아닌 리스트의 연결값이 출력됨

```
중간시험과 기말시험 성적의 합 : [70, 85, 92, 79, 90, 65, 70, 85]
```

```
student_sum / 2    # 오류! : 리스트는 나누기 연산을 지원하지 않음
```

```
    student_sum / 2
TypeError: unsupported operand type(s) for /: 'list' and 'int'
```

- 넘파이의 강력한 다차원배열

```
# 넘파이의 다차원 배열 연산
student_mid = np.array([70, 85, 92, 79])  # 중간시험 성적
student_fin = np.array([90, 65, 70, 85])  # 기말시험 성적
student_sum = student_mid + student_fin
print('중간시험과 기말시험 성적의 합 :', student_sum)
```

```
중간시험과 기말시험 성적의 합 : [160 150 162 164]
```

+ 연산으로 중간시험과 기말시험 개별 합이 구해진다

```
print('평균 성적 :', student_sum/2)
```

```
평균 성적 : [80. 75. 81. 82.]
```

/2 연산으로 중간시험과 기말시험의 평균을 구할 수 있음

다차원 배열의 연산

- 리스트를 넘파이의 다차원 배열로 고치기
  - 다차원 배열에 대해 사칙 연산을 하면 개별 성분에 대한 연산이 이루어진다.

```
a = np.array([10, 20, 30])      # 넘파이 ndarray 객체 a 생성
b = np.array([1, 2, 3])         # 넘파이 ndarray 객체 b 생성
a + b
```

```
array([11, 22, 33])
```

```
a - b
```

```
array([ 9, 18, 27])
```

```
a * b
```

```
array([10, 40, 90])
```

```
a / b
```

```
array([10., 10., 10.])
```

다차원 배열의 연산

도전 문제 3.4: 다차원 배열 다루기

상 중 하



1. 0에서 9까지의 정수 값을 가지는 ndarray 객체 a를 넘파이를 이용하여 작성하여 다음과 같이 출력하여라.

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

2. 문제 1의 코드를 수정하여 0에서 9까지의 정수 값 중에서 다음과 같이 짝수를 가지는 ndarray 객체 b를 출력하여라.

```
array([0, 2, 4, 6, 8])
```

3. 문제 2번 ndarray 객체 b의 shape, ndim, dtype, size, itemsize를 다음과 같이 출력하여라.

```
b.shape = (5,)
b.ndim = 1
b.dtype = int64
b.size = 5
b.itemsize = 8
```

편리하고 강력한 브로드캐스팅과 벡터화 연산

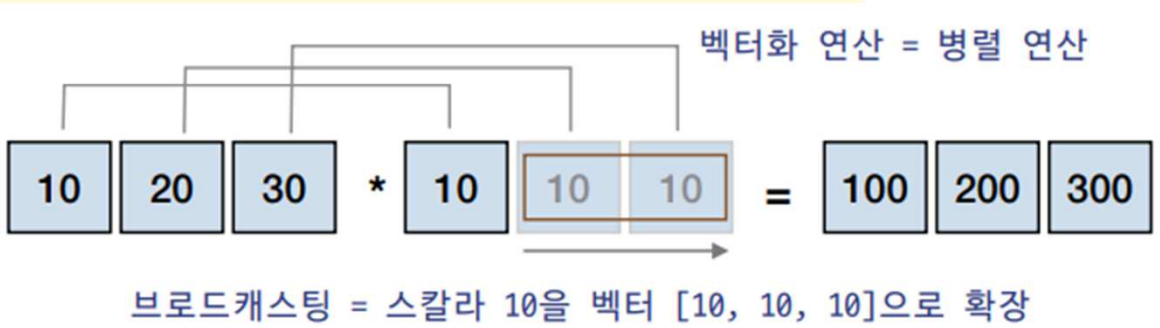
- 브로드캐스팅과 벡터화 연산
  - 넘파이는 a * 10을 수행할 때 np.array([10, 20, 30]) * np.array([10, 10, 10])과 같이 a
  - 의 차원에 맞게, 그림과 같이 스칼라 10을 벡터로 확장시켜주는 작업을 하는데 이를 **브로드캐스팅** broadcasting 이라고 한다.
  - 브로드캐스팅과 동시에 파이썬은 하나의 명령을 여러 데이터에 적용하여 병렬적으로 연산하는데 이것을 **벡터화 연산** vectorized operation 이라고 한다
    - 빠르고 효율적인 연산기능을 제공

```
a = np.array([10, 20, 30])
a * 10
```

```
array([100, 200, 300])
```

```
a + 10
```

```
array([20, 30, 40])
```



편리하고 강력한 브로드캐스팅과 벡터화 연산

- 매우 편리한 브로드캐스팅과 벡터화 연산

```
b = np.array([[10, 20, 30],
              [40, 50, 60]])
c = np.array([2, 3, 4])
b + c
```

```
array([[12, 23, 34],
       [42, 53, 64]])
```

```
b * c
```

```
array([[ 20,  60, 120],
       [ 80, 150, 240]])
```



다차원 배열을 쉽게 생성해 보자

```
np.array(range(1, 11))          # 파이썬의 range() 함수로 연속된 수를 만든다
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
np.zeros((2, 3))                # 2행 3열의 행렬 생성 시 모든 값을 0으로
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
np.ones((2, 3))                 # 2행 3열의 행렬 생성 시 모든 값을 1로
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
np.full((2, 3), 100)            # 2행 3열의 행렬 생성 시 모든 값을 100으로
```

```
array([[100, 100, 100],
       [100, 100, 100]])
```

```
np.eye(3)                       # 3x3 크기의 단위행렬 생성
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

다차원 배열을 쉽게 생성해 보자

도전 문제 3.5: 다차원 배열 생성하기

상 중 하



대각 성분이 모두 10인 4×4 크기의 다차원 배열을 생성하여라.

```
array([[10.,  0.,  0.,  0.],
       [ 0., 10.,  0.,  0.],
       [ 0.,  0., 10.,  0.],
       [ 0.,  0.,  0., 10.]])
```

다차원 배열을 합하는 append() 함수

- append() 함수는 인자로 입력된 두 배열을 합하는 일을 하는데 axis를 통해서 축을 명시하지 않으면 디폴트 형상인 1차원 배열로 변환하게 된다

```
a = np.array([1, 2, 3])
b = np.array([[4, 5, 6], [7, 8, 9]])
np.append(a, b)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.append([a], b, axis = 0)          # [a]를 통해 2차원 배열로 만들어야 함
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

다차원 배열을 합하는 append() 함수

- 다차원배열 a, b에 대하여 a * b를 구하면 a, b의 각 원소끼리의 곱을 단순 계산한다.
  - 넘파이에서 제공하는 행렬 곱 함수는 matmul()이다. 그리고 이 matmul() 함수를 조금 더 편리하게 사용할 수 있는 것이 @ 연산자이다.

```
a = np.array([[1, 2], [3, 4]])          # 2차원 배열 a
b = np.array([[10, 20], [30, 40]])      # 2차원 배열 b
a * b                                   # 2차원 배열의 곱하기 연산 - 원소 간의 곱셈
```

```
array([[ 10,  40],
       [ 90, 160]])
```

```
np.matmul(a, b)       # 2차원 배열 a, b의 행렬곱, a @ b와 같다
```

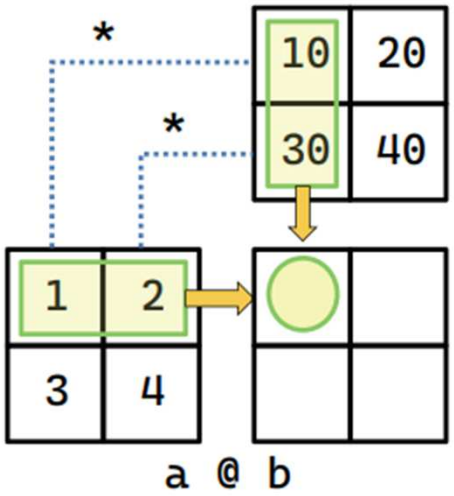
```
array([[ 70, 100],
       [150, 220]])
```

```
a @ b                 # 2차원 배열 a, b의 행렬곱, 위의 결과와 동일함
```

```
array([[ 70, 100],
       [150, 220]])
```

다차원 배열을 합하는 append() 함수

- 행렬곱연산의 과정



: 1 * 10 + 2 * 30

행렬곱 연산은 a 행렬과 b 행렬의 동일한 행과 열 성분을 곱하여 더한다.
따라서 a 행렬의 행 크기와 b 행렬의 열의 크기가 같아야만 한다.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 * 10 & 2 * 20 \\ 3 * 30 & 4 * 40 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 * 10 + 2 * 30 & 1 * 20 + 2 * 40 \\ 3 * 10 + 4 * 30 & 3 * 20 + 4 * 40 \end{bmatrix}$$

다차원 배열을 합하는 append() 함수

도전 문제 3.6: 다차원 행렬의 생성과 활용

상 중 하



1. 1에서 9까지의 모든 정수 값을 크기 순서대로 가지는 3×3 크기의 행렬 a를 arange()와 reshape()을 이용하여 생성하고, 모든 성분의 값이 3인 3×3 크기의 행렬 b를 full() 함수를 사용하여 생성한 후, 다음과 같이 출력하여라. reshape(3, 3) 메소드는 3행 3열의 다차원 행렬을 만드는 함수이다.

```
a = [[1 2 3]                    b = [[3 3 3]
     [4 5 6]                         [3 3 3]
     [7 8 9]]                        [3 3 3]]
```

2. 다음과 같은 행렬 연산을 수행한 후 그 결과를 출력하여라.

```
a + b, a - b, a * b, a / b, a @ b, a ** 2
```

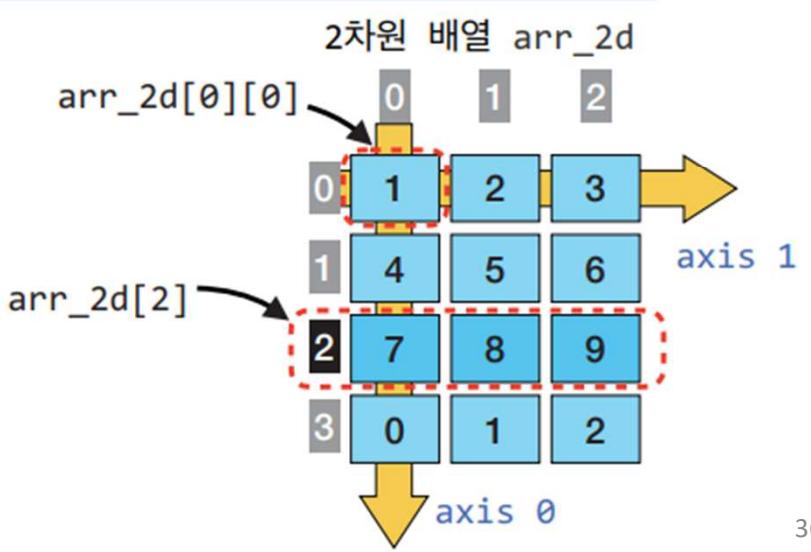
04 넘파이의 고급 기능을 알아보자

- 인덱싱과 슬라이싱
  - 다차원 배열 내의 원소는 **인덱스** index 라고 불리는 정수를 사용하여 참조할 수 있다.
  - 리스트에서 사용했던 인덱싱과 슬라이싱은 넘파이 배열에도 거의 동일하게 적용된다.

```
arr_2d = np.array( [[1, 2, 3], [4, 5, 6],
                    [7, 8, 9], [0, 1, 2]] )
```

```
# 첫 번째 행의 첫 번째 데이터
print(arr_2d[0][0])
# 세 번째 열의 데이터
print(arr_2d[2])
```

```
1
[7, 8, 9]
```



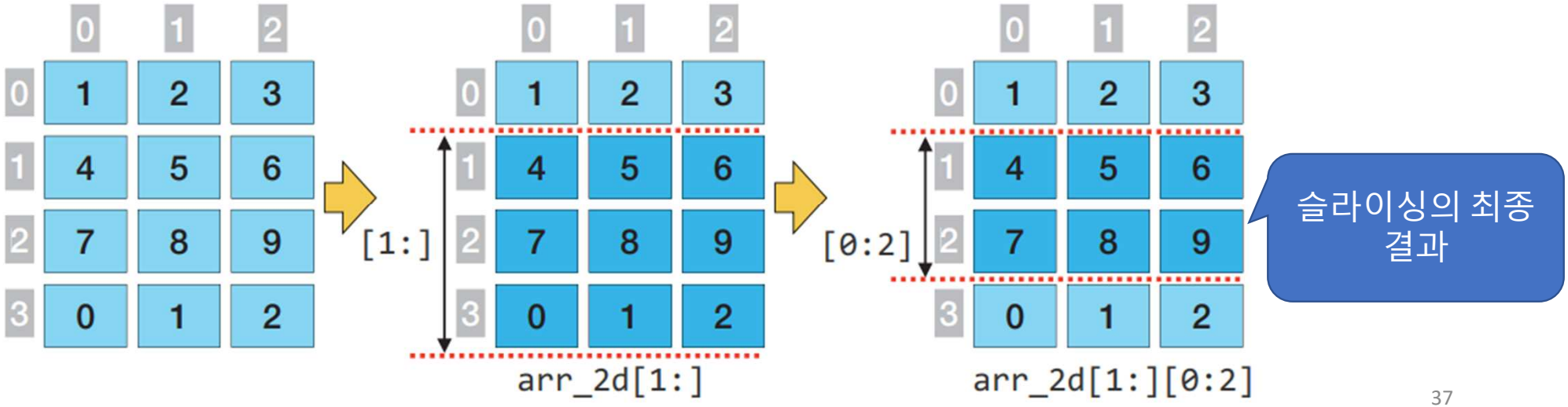
인덱싱과 슬라이싱

- 슬라이싱 : 행렬의 특정한 항목을 구간을 정해서 추출하는 기능

```
arr_2d[1:][0:2]
```

```
array([[4, 5, 6],
       [7, 8, 9]])
```

슬라이싱 과정



인덱싱과 슬라이싱

- 선형대수에서는 가로로 나열된 행과 세로로 나열된 열에 대해서 각각 슬라이싱을 해서 일부를 가져오는 일이 매우 빈번히 요구된다.
  - 넘파이는 이러한 요구에 맞는 넘파이만의 인덱싱 방법이 있다.
  - [i][j]와 같이 원소를 인덱싱하는 것이 아니라 [i, j]로 인덱싱하는 방법을 제공.
  - 각 축의 길이를 파악하여 length0, length1에 저장하고, 이 값의 범위 내에서 인덱스 i, j를 바꾸어 [i][j] 인덱싱과 [i,j] 인덱싱을 비교해 보자. 동일한 결과가 나온다.

```
length0, length1 = arr_2d.shape
for i in range(length0):
    for j in range(length1):
        print(arr_2d[i][j], arr_2d[i, j]) # 두 방식의 결과는 동일함
```

```
1 1
2 2
...
1 1
2 2
```

인덱싱과 슬라이싱

- **논리 인덱싱** logical indexing 이란 넘파이의 배열에서 특정한 조건을 주고, 이 조건을 통해서 배열에서 원하는 값을 추려내는 것
  - 다차원 배열에서 21이상의 원소만을 추출하는 방법을 알아보자

```
ages = np.array([28, 25, 12, 16, 20])
```

```
y = ages >= 21
y
```

```
array([ True,  True, False, False, False])
```

```
age_over_21 = ages[ ages >= 21 ]
print(f'age_over_21 = {age_over_21}')
```

```
age_over_21 = [28 25]
```

난수를 생성해 보자

- 넘파이는 풍부한 난수 생성 기능을 제공
  - 넘파이 모듈의 random 서브 모듈에서 제공하는데 randn() 함수는 표준 정규분포를 따르는 난수를 생성
    - 데이터들의 평균값이 165, 표준편차가 10인 정규분포를 따른다고 할 경우 5개의 난수를 생성하자

```
rnd = np.random.randn(5) * 10 + 165 # 평균값이 165, 표준편차가 10인 값 생성
rnd
```

```
array([176.07979332, 177.91821264, 165.45715947, 164.34633461,
       167.9482772 ])
```

```
rnd.round(2)        # 소수점 아래 둘째 자리까지 값으로 변환
```

```
array([176.08, 177.92, 165.46, 164.35, 167.95])
```

```
rnd.astype(int)     # 정수형 자료형으로 변환
```

```
array([176, 177, 165, 164, 167])
```

난수를 생성해 보자

- 난수의 시드는 초기값을 결정한다.
  - 따라서 난수의 열이 매번 동일한 값을 생성하도록 하기 위해서는 시드를 동일하게 하면된다.
  - seed() 함수를 사용한다.
  - seed(42)와 같이 특정한 값을 사용하면 된다

```
np.random.seed(42)        # 시드값이 특정되면 매번 동일한 난수의 열이 생성됨
```

난수를 생성해 보자

- 정규분포는 매우 중요한 난수로 normal() 함수를 사용해서 얻을 수도 있다.
- 평균값이 165, 표준편차가 10인 정규분포 함수는 다음과 같이 생성 가능하다.
  - shape 키워드 인자로 (3, 4)와 같이 3행 4열의 난수 생성 가능
  - .round(2) 메소드로 소수점 아래 둘째자리까지 생성

```
nums = np.random.normal(loc=165, scale=10, size=(3, 4)).round(2)
nums
```

```
array([[169.97, 163.62, 171.48, 180.23],
       [162.66, 162.66, 180.79, 172.67],
       [160.31, 170.43, 160.37, 160.34]])
```

난수를 생성해 보자

- 중요한 난수 관련 함수

함수 이름	설명
rand()	균등분포의 난수 표본을 추출한다.
randn()	표준정규분포의 난수 표본을 추출한다.
randint()	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성한다. 이때 최대값은 포함하지 않는다.
normal()	평균과 표준편차를 인자로 받아, 정규분포의 난수 표본을 추출한다.
uniform()	균등분포에서 표본을 추출한다.
shuffle()	주어진 리스트나 다차원 배열의 순서를 임의로 섞는다.
permutation()	주어진 범위의 수를 생성하여 섞거나, 배열을 임의의 순서로 섞는다.

난수를 생성해 보자

- 난수를 생성하여 섞어보자.

```
a = np.arange(10)       # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
a                       # a에는 순서가 뒤섞인 값이 들어있다
```

```
array([2, 8, 9, 6, 4, 0, 1, 5, 7, 3])
```

```
# permutation() 함수는 리스트 원소를 랜덤한 순서로 재배치한 후
# ndarray로 반환한다
np.random.permutation([2, 4, 6, 8, 10])
```

```
array([ 8,  4, 10,  6,  2])
```

난수를 생성해 보자

- 10개의 수를 생성하자
  - 이를 랜덤하게 섞자
  - 50:50의 비율로 균등하게 나누어보자
  - 딥러닝에 필요한 훈련 데이터와 테스트 데이터를 나누는 기능은 이렇게 동작한다

```
a = np.arange(10)                # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
print('50% 데이터 :', a[:5])              # 순서가 뒤섞인 값이 5개 들어있다
print('나머지 50% 데이터 :', a[5:])       # 순서가 뒤섞인 값이 5개 들어있다
```

```
50% 데이터 : [2 7 4 0 6]
나머지 50% 데이터 : [1 5 9 8 3]
```

난수를 생성해 보자

도전 문제 3.7: 다차원 배열 만들기

상 중 하



arange()를 사용하여 1에서 50까지의 원소를 가지는 다차원 배열을 만들자. 이 배열의 원소 50개를 랜덤하게 섞은 후 80%의 데이터는 train_data에 넣고 나머지 20%의 데이터는 test_data라는 배열에 넣어서 이 두 배열을 반환하는 함수 train_test_split()을 만들자. 반환된 배열 값을 각각 다음과 같이 출력하여라.

```
train_data = [36  7 30 15 14  8 21 13 10 31]
test_data = [49 22 38 12  9 39 18 16 29 25 42 20  5 44 34 47 27 24 11 40  4
             43 35  6 41 26 45 19  1 33 32 48 23 28 46 50 17 37  3  2]
```

여러 개의 배열값을 얻는 방법

- 여러 개의 값을 쉽게 생성하는 방법
  - linspace()는 시작값부터 마지막 값까지 균일한 간격으로 지정된 수만큼의 배열을 생성한다.
  - 이때 그림과 같이 마지막 값은 생성될 데이터 값에 포함된다.
  - 데이터의 수는 num이라는 키워드 인자로 지정하며 디폴트로 50개가 생성된다



정수가 아니라 실수 데이터가 생성된다.
start에서 stop까지의 간격을 균일하게 쪼개어 num개의 실수를 생성한다.

여러 개의 배열값을 얻는 방법

- linspace(0, 10, 100)이라고 호출하면 0에서 10까지 총 100개의 수들이 생성된다.

```
np.linspace(0, 10, 100)
```

```
array([ 0.        ,  0.1010101 ,  0.2020202 ,  0.3030303 ,  0.4040404 ,
            ...
        8.08080808,  8.18181818,  8.28282828,  8.38383838,  8.48484848,
        8.58585859,  8.68686869,  8.78787879,  8.88888889,  8.98989899,
        9.09090909,  9.19191919,  9.29292929,  9.39393939,  9.49494949,
        9.5959596 ,  9.6969697 ,  9.7979798 ,  9.8989899 , 10.        ])
```

여러 개의 배열값을 얻는 방법

- 넘파이 함수로 logspace() 함수가 있다.
  - 이것은 로그 스케일로 수들을 생성한다. 형식은 logspace(x, y, num)이며, 생성되는 수의 시작은 10^x부터 10^y까지 생성된다

```
np.logspace(0, 5, 10)
```

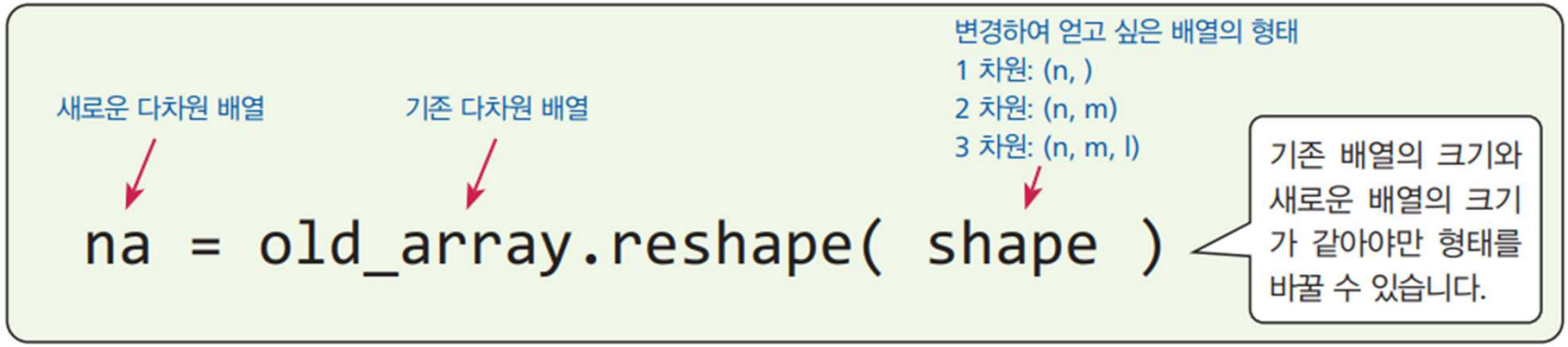
```
array([1.00000000e+00, 3.59381366e+00, 1.29154967e+01, 4.64158883e+01,
       1.66810054e+02, 5.99484250e+02, 2.15443469e+03, 7.74263683e+03,
       2.78255940e+04, 1.00000000e+05])
```



10^{start}부터 10^{stop}까지의 실수를 로그 스케일로 볼 때 균등한 간격으로 num개 생성한다.
여기서는 10을 밑으로 잡았지만, base 키워드 매개변수에 설정한 인자에 따라 바꿀 수도 있다.

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- reshape()은 데이터의 개수는 유지한 채로 배열의 차원과 형태를 변경한다.
  - 이 함수의 인자인 shape을 튜플의 형태로 넘겨주는 것이 원칙이지만 reshape(x, y)라고 하면 reshape((x, y))와 동일하게 처리됨



배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- arange(12)를 통해 생성한 y는 0에서 11까지 12개의 원소를 가지는 배열이 된다.
  - 이 y에 대하여 reshape(3, 4)를 호출하면, 1차원 배열이 3행 4열의 2차원 배열로 바뀌게 된다.

```
y = np.arange(12)    # 0에서 11까지 12개의 수를 생성한다
y
```

```
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
y.reshape(3, 4)      # 다차원 배열의 shape을 (3, 4)로 만든다
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- reshape() 메소드의 첫 번째 인자로 –1을 전달하면 두 번째 인자와 데이터의 개수에 맞춰서 자동으로 배열의 형태가 결정된다.
- -1은 NumPy가 해당 차원의 크기를 자동으로 계산하라는 특별한 값이다.

```
y.reshape(-1, 4)      # 다차원 배열의 shape을 (3, 4)로 만든다
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- 배열의 차원을 3차원으로 만들 때는 다음과 같이 세 개의 튜플을 reshape()의 인자로 넣으면 된다.
  - 다음과 같은 (-1, 2, 3) 입력은 2행 3열 배열을 두 개 생성한다.

```
# 3차원 배열을 만든다
y.reshape(-1, 2, 3)
```

```
array([[[ 0,  1,  2],
        [ 3,  4,  5]],

       [[ 6,  7,  8],
        [ 9, 10, 11]]])
```

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- 배열의 구조를 그대로 두고 차원을 증가시키기 위해서는 np.newaxis를 기존 다차원 배열의 새 원소로 넣어 주면 된다

```
y = np.arange(3)      # 1차원 배열
y = y[:, np.newaxis] # 차원을 2차원으로 만든다
y
```

```
array([[0],
       [1],
       [2]])
```

```
y = np.arange(3)                        # 1차원 배열
y = y[:, np.newaxis, np.newaxis]   # 차원을 3차원으로 만든다
y
```

```
array([[[0]],
       [[1]],
       [[2]]])
```

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

- 다차원 배열이 2차원 이상의 배열이라면 이를 1차원 배열로 변환할 필요성도 있는데 이때 사용하는 메소드가 바로 flatten() 메소드이다

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.flatten()    # 넘파이 다차원 배열의 평탄화 메소드
```

```
array([1, 1, 2, 2, 3, 3])
```

- 2차원 배열의 행과 열을 서로 교환하는 것을 전치(transpose) 라고 하는데 다차원 배열의 전치는 .T 속성값을 이용하면 된다.

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.T
```

```
array([[1, 2, 3],
       [1, 2, 3]])
```

배열의 형태를 바꾸는 reshape() 메소드와 flatten() 메소드

도전 문제 3.8: 다차원 배열 만들기

상 중 하



1. 0부터 9까지의 연속된 수를 가진 다차원 배열 a를 만들어라.

```
a = [0 1 2 3 4 5 6 7 8 9]
```

2. a를 2행 5열의 형태를 가지는 다차원 배열로 만들어라.

```
a = [[0 1 2 3 4]
     [5 6 7 8 9]]
```

3. 이 행렬의 전치행렬을 만들어서 다음과 같이 출력하여라.

```
[[0 5]
 [1 6]
 [2 7]
 [3 8]
 [4 9]]
```

- 딥러닝을 위한 행렬의 핵심 개념
  - 임의의 벡터 u = (1, 2, 0, −1)과 v = (2, 2, 3, 4)가 있다고 할 때, 이 벡터는 다음과
  - 같은 $R^{n \times 1}$ 행렬로 표현할 수 있다

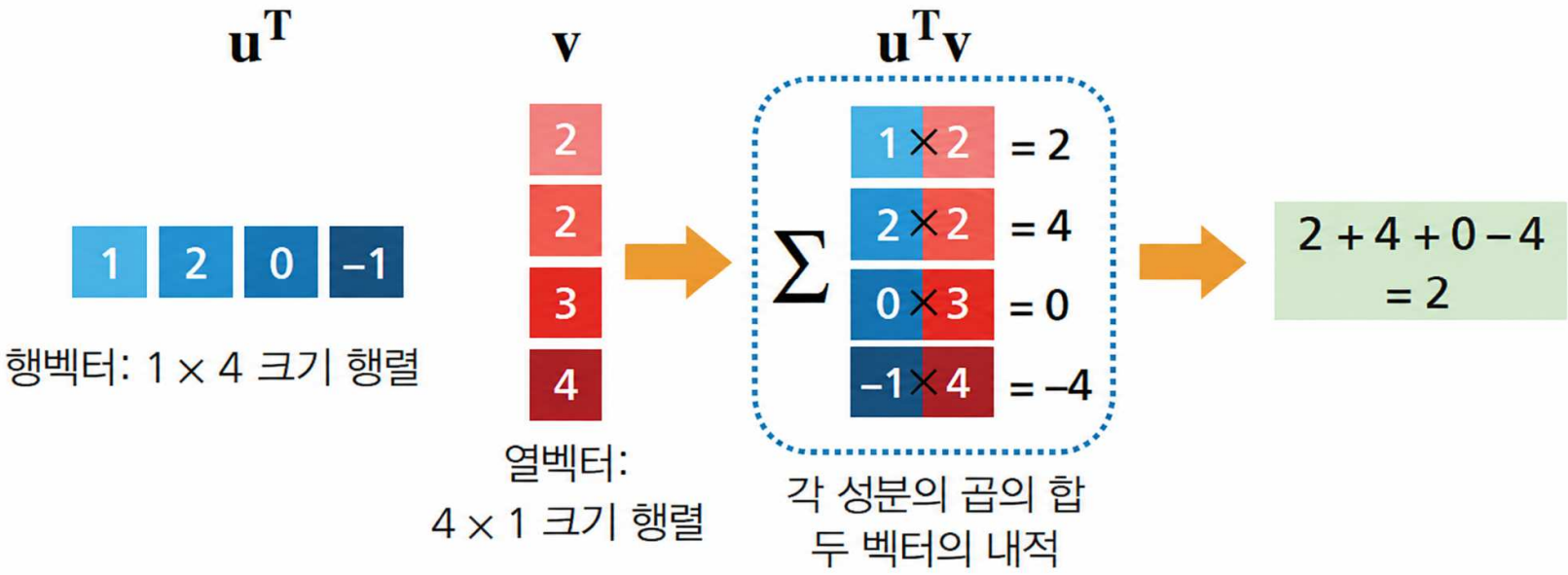
$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}, \mathbf{v} = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

  - 두 벡터의 내적은 전치행렬을 만들어 행렬곱을 하면 된다.

$$\mathbf{u}^T\mathbf{v} = \begin{bmatrix} 1 & 2 & 0 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 3 \\ 4 \end{bmatrix} = 2 = \mathbf{u} \cdot \mathbf{v}$$

딥러닝을 위한 행렬의 핵심 개념

- 다음과 같이 행렬의 곱셈에서 행 성분과 열 성분의 곱의 합을 구하는 방식과 동일하다.



딥러닝을 위한 행렬의 핵심 개념

- 넘파이로 벡터의 내적 구현하기

```
a = np.array([1, -4])
b = np.array([3, 6])
print('a =', a)
print('b =', b)
```

```
a = [ 1 -4]
b = [3 6]
```

```
print('a dot b = ', np.dot(a, b))
```

```
a dot b =  -21
```

```
print('a dot b = ', a.dot(b))
```

```
a dot b =  -21
```

딥러닝을 위한 행렬의 핵심 개념

- 두 벡터의 성분별 곱은 u * v를 이용하여 구할 수 있다.
  - 이 성분들의 합을 np.sum()을 이용해서 구하는 과정은 np.dot(u, v)을 사용하는 것과 동일한 결과를 낸다

```
u = np.array([1, 2, 0, -1])
v = np.array([2, 2, 3, 4])
print('u dot v = ', np.dot(u, v))
```

```
u dot v =  2
```

```
print('u * v =', u * v)
```

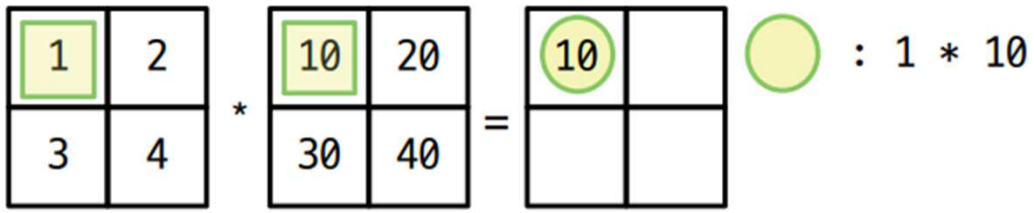
```
u * v = [ 2  4  0 -4]
```

```
print('np.sum(u * v) =', np.sum(u * v))
```

```
np.sum(u * v) = 2
```

행렬 곱 연산

- 행렬의 성분별 곱과 그 결과



a * b의 계산 과정

```
a = np.array([[1, 2], [3, 4]])      # 2차원 배열 a
b = np.array([[10, 20], [30, 40]]) # 2차원 배열 b
a * b        # 행렬 간의 원소 곱 연산자 *를 사용해 보자
```

```
array([[ 10,  40],
       [ 90, 160]])
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 \times 10 & 2 \times 20 \\ 3 \times 30 & 4 \times 40 \end{bmatrix} = \begin{bmatrix} 10 & 40 \\ 90 & 160 \end{bmatrix}$$

행렬 곱 연산

- 행렬곱의 연산과 그 결과
  - matmul() 메소드 또는 @ 연산자를 이용한다

```
a = np.array([[1, 2], [3, 4]])      # 2차원 배열 a
b = np.array([[10, 20], [30, 40]]) # 2차원 배열 b
np.matmul(a, b)
```

```
array([[ 70, 100],
       [150, 220]])
```

```
a @ b     # 행렬 곱 연산자 @를 사용해 보자
```

```
array([[ 70, 100],
       [150, 220]])
```

행렬 곱 연산

- 행렬곱의 과정

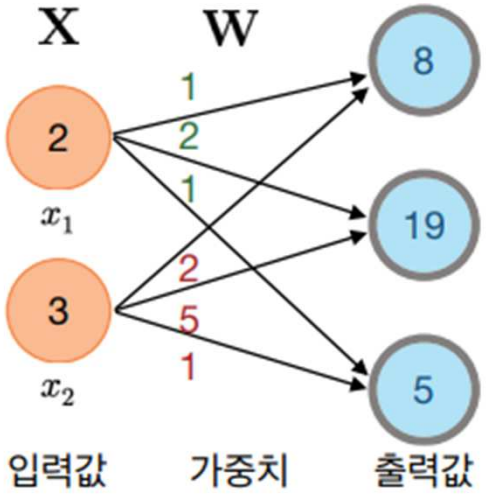
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1 \times 10 + 2 \times 30 & 1 \times 20 + 2 \times 40 \\ 3 \times 10 + 4 \times 30 & 3 \times 20 + 4 \times 40 \end{bmatrix}$$

```
a = np.array([[1, 2], [3, 4]])      # 2차원 배열 a
b = np.array([[10, 20], [30, 40]]) # 2차원 배열 b
np.matmul(a, b)
```

```
array([[ 70, 100],
       [150, 220]])
```

인공신경망과 행렬

- 딥러닝의 주제인 인공신경망은 내부에서 대규모의 행렬 연산을 사용한다
  - 퍼셉트론이란 인간의 신경세포를 컴퓨터로 구현한 것으로 이웃한 신경세포에 자극을 전달하는 기능을 모방하고 있다
  - 동일한 입력이 있더라도 가중치 값이 달라지면 그 출력 결과는 달라진다. 이 가중치는 출력을 결정짓는 값으로 **파라미터** parameter 라고 부르기도 한다
  - 수학 분야 중에서 선형대수가 딥러닝의 기초가 되는 학문이다.



행렬의 형태

$$(1 \times 2) * (2 \times 3) = (1 \times 3)$$

입력 행렬 가중치 행렬 출력 행렬

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 1 \end{bmatrix} = \begin{bmatrix} 8 & 19 & 5 \end{bmatrix}$$

출력 결과

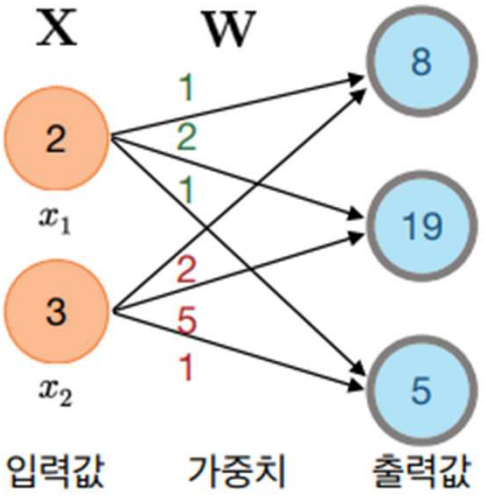
인공신경망과 행렬

- 다음과 같은 입력 X, 가중치 W가 있을 경우 이들의 합이 결과값 A를 만든다

$$X = [x_1 \;\; x_2]$$

$$W = \begin{bmatrix} \omega_{11} & \omega_{21} & \omega_{31} \\ \omega_{12} & \omega_{22} & \omega_{32} \end{bmatrix}$$

$$A = XW$$



행렬의 형태

$(1 \times 2) * (2 \times 3) = (1 \times 3)$

입력 행렬 가중치 행렬 출력 행렬

$$[2 \;\; 3] \begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 1 \end{bmatrix} = [8 \;\; 19 \;\; 5]$$

출력 결과

인공신경망과 행렬

- 이 과정을 넘파이로 구현할 수 있다.
  - 넘파이의 다차원 배열은 텐서플로에서 내부적으로 사용하는 자료구조이다

```
X = [2, 3]
W = [[1, 2, 1], [2, 5, 1]]      # X는 (2, 3)크기의 행렬
np.matmul(X, W)                 # X 행렬과 W 행렬의 곱의 결과
```

```
array([ 8, 19,  5])
```

단위행렬과의 곱

- 행렬이 행과 열의 크기가 같은 정사각행렬이고 정사각행렬의 모든 대각선 성분이 1이며, 그 외의 성분이 0인 행렬을 **단위행렬** identity matrix 이라고 한다.
  - 정사각행렬 a와 곱할 b 행렬이 단위행렬일 경우 원래 행렬 a를 반환한다.

```
a = [[1, 2], [3, 4]]
b = [[1, 0], [0, 1]]  # b는 단위행렬
np.matmul(a, b)       # a 행렬과 단위행렬 b의 곱의 결과
```

```
array([[1, 2],
       [3, 4]])
```

```
c = [[1, 2, -1], [3, 4, 5], [2, 1, 0]]
d = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]  # d는 (3, 3)크기의 단위행렬
np.matmul(c, d)                        # c 행렬과 단위행렬 d의 곱의 결과
```

```
array([[ 1, 2, -1],
       [ 3, 4,  5]
       [ 2, 1,  0]])
```

단위행렬과의 곱

- 다음 문제를 통해서 행렬 곱에 대한 개념을 학습하자

도전 문제 3.9: 행렬의 곱셈

상 중 하



1. 다음과 같은 행렬 A, B, C, D가 있다. 이 행렬에 대하여 다음과 같은 행렬 곱셈을 넘파이를 이용하여 수행하여라.

$$A = \begin{bmatrix} 1 & -2 \\ 3 & -1 \end{bmatrix}, B = \begin{bmatrix} 2 & -3 & 4 \\ 0 & -1 & 2 \end{bmatrix}, C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, D = \begin{bmatrix} 0 & 6 & 4 \\ 3 & 2 & 1 \\ 2 & 1 & 0 \end{bmatrix}$$

1) AA 2) AB 3) AC 4) CC 5) CB 6) BD 7) DD

요약

01 딥러닝을 익히기 위하여 사용되는 인기 있는 프로그래밍 언어인 파이썬은 정수, 부동소수점, 문자와 같은 전통적인 **자료형**뿐만 아니라, 데이터 묶음을 처리하기에 편리한 **리스트**, **튜플**, **딕셔너리**, **집합**과 같은 자료형을 기본으로 제공한다.

02 특정한 클래스에 속한 객체들이 사용할 수 있는 함수들을 해당 클래스의 **메소드**라고 부른다.**객체 지향 언어**는 리스트와 같은 클래스를 정의하고, 해당 클래스의 인스턴스 객체를 생성한 뒤, 이들이 메소드를 이용하여 다양한 일을 할 수 있도록 지원하는 언어를 의미한다.

03 **코랩** 환경은 파이썬의 다양한 **데이터 처리 패키지** **넘파이**, **판다스**와 **사이킷런**과 **텐서플로**와 같은 **머신러닝 도구**를 기본적으로 지원한다.

04 **텐서플로**는 **머신러닝과 딥러닝을 위한 오픈 소스 플랫폼**으로 구글의 인공지능 개발부서에서 개발하여 내부적으로 사용하다가 2015년에 오픈 소스로 공개되어 2019년 텐서플로 2 버전으로 발전하였다.

05 **넘파이**는 매우 빠르고 편리하게 수치 데이터를 처리할 수 있기 때문에 파이썬에서 수치 데이터를 다루는 가장 기본적이고 강력한 패키지라 할 수 있다.

06 **넘파이 배열**의 개별적인 값을 **항목** 또는 **원소**라 하며, 연속적인 원소들을 가지는 객체가 넘파이의 핵심이 되는 **다차원 배열** 객체이다. 이 배열 내부의 각 요소는 **인덱스**라고 불리는 정수들로 참조되며, 넘파이에서 차원은 **축**이라고도 한다.

07 다차원 배열 내의 원소는 **인덱스**라고 불리는 정수를 사용하여 참조할 수 있으며, 리스트에서 사용했던 인덱싱과 슬라이싱은 넘파이 배열에도 거의 동일하게 적용된다.

08 넘파이는 데이터를 추려내는 데에 슬라이싱뿐만 아니라 **논리 인덱싱**이라는 효율적인 방법도 제공한다. 이것은 특정한 어떤 조건을 주어서 배열에서 **원하는 값을 추려내는 것이다**.

09 인공신경망을 구현하기 위해서는 **행렬**이라는 수학 도구를 사용하는 것이 매우 편리하다.